

Taller WAAP — Informe de laboratorio Fase 0 a Fase 2

Prototipo WAAP (OWASP Juice Shop 20.2.0 + ModSecurity CRS 3.3.10).

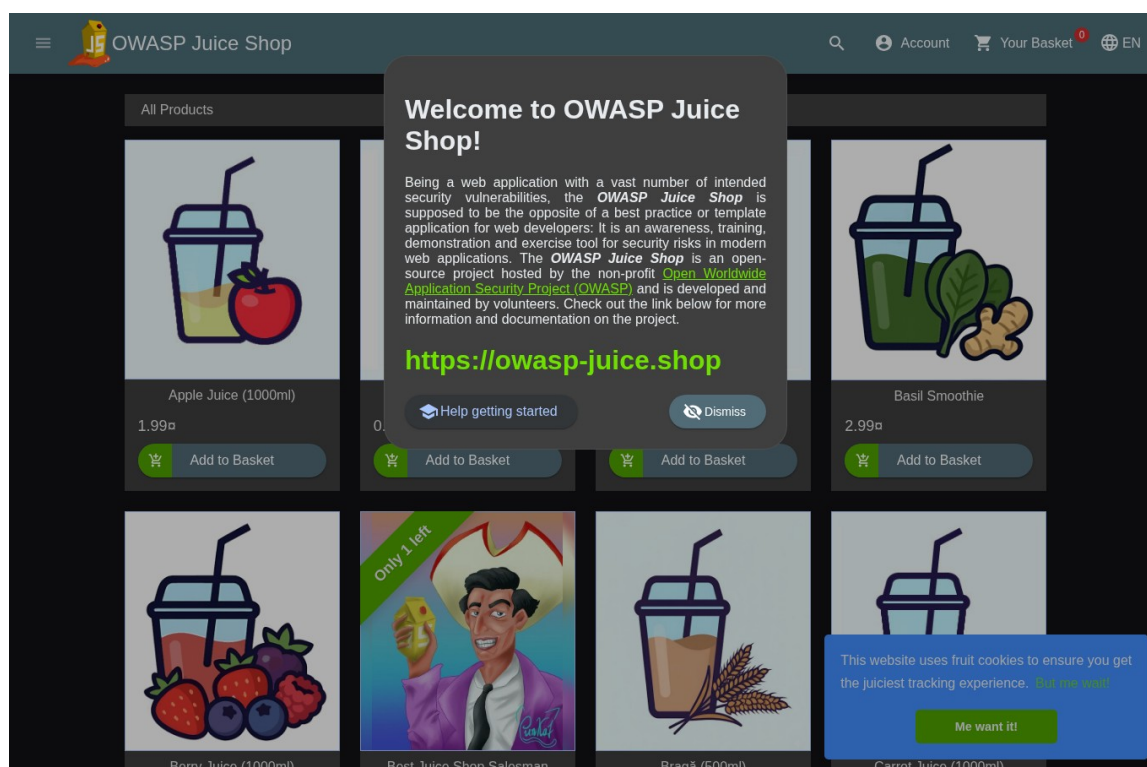
Entorno: Ubuntu 22.04, Docker + Docker Compose. Acceso vía WAAP:

<http://localhost:8080> (backend directo <http://localhost:3000>). Nivel de paranoia CRS: PL1 (PARANOIA=1).

Fase 0 — Preparación del entorno

Se copió `.env.example` a `.env` (JUICE_PORT=3000, WAAP_PORT=8080, PARANOIA=1) y se levantó docker compose (juice-shop + waap-proxy en red lab-net). Ambos contenedores Up (waap-proxy healthy). Verificación: curl a <http://localhost:8080> responde 200 OK con Server: nginx (proxy WAAP).

Figura 1. Juice Shop a través del proxy WAAP (puerto 8080).



Fase 1 — Aplicación objetivo vulnerable

OWASP Juice Shop 20.2.0 desplegado. Score Board disponible en `/#/score-board` (categorías OWASP: Injection, Broken Access Control, XSS, Sensitive Data Exposure, etc.). El endpoint `/rest/products/search?q=apple` responde JSON success a través del WAAP.

Figura 2. Score Board con retos por categoría OWASP.

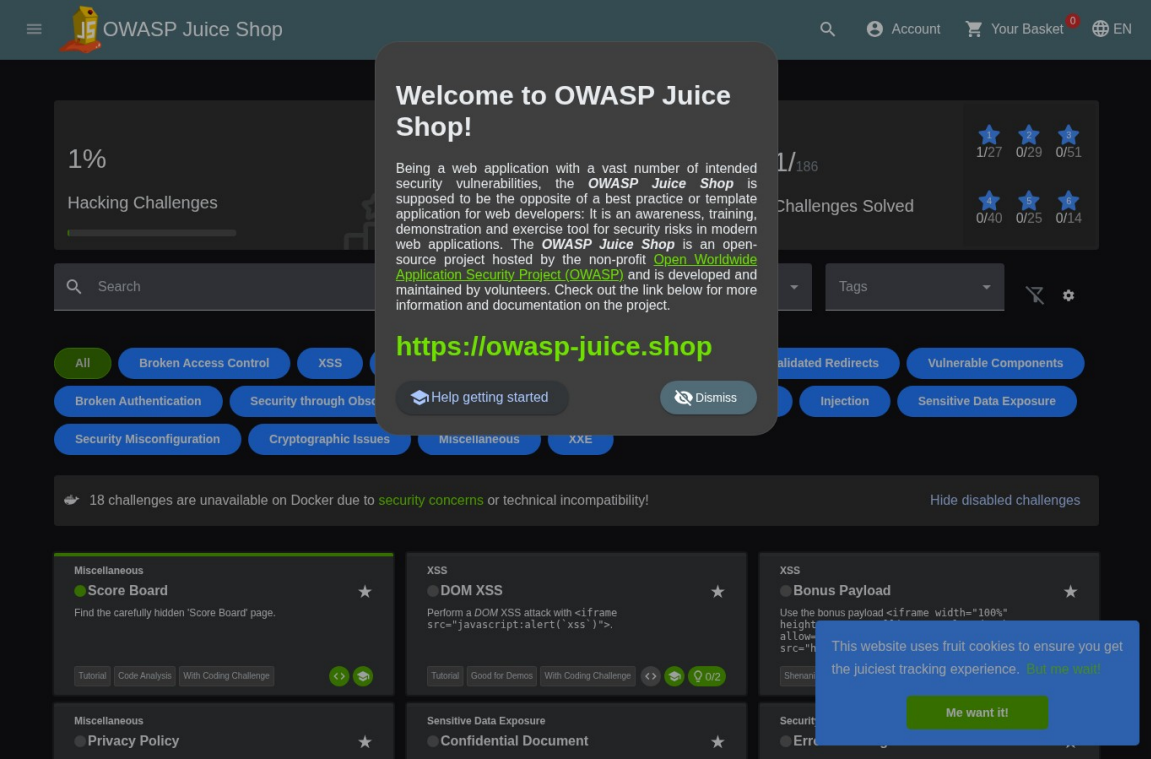


Figura 3. Tráfico normal: búsqueda de producto vía WAAP (200).

```

Pretty-print
{
  "status": "success",
  "data": [
    {
      "id": 1,
      "name": "Apple Juice (1000ml)",
      "description": "The all-time classic.",
      "price": 1.99,
      "deluxePrice": 0.99,
      "image": "apple_juice.jpg",
      "createdAt": "2026-09-17 23:55:44.453 +00:00",
      "updatedAt": "2026-09-17 23:55:44.453 +00:00",
      "deletedAt": null
    },
    {
      "id": 24,
      "name": "Apple Pomace",
      "description": "Finest pressings of apples. Allergy disclaimer: Might contain traces of worms. Can be <a href=\"/#recycle\">sent back to us</a> for recycling.",
      "price": 0.89,
      "deluxePrice": 0.89,
      "image": "apple_pressings.jpg",
      "createdAt": "2026-09-17 23:55:44.462 +00:00",
      "updatedAt": "2026-09-17 23:55:44.462 +00:00",
      "deletedAt": null
    },
    {
      "id": 47,
      "name": "Pineapple Juice (1000ml)",
      "description": "Tropical refreshment from the finest sun-ripened pineapples.",
      "price": 2.99,
      "deluxePrice": 2.99,
      "image": "pineapple_juice.png",
      "createdAt": "2026-09-17 23:55:44.464 +00:00",
      "updatedAt": "2026-09-17 23:55:44.464 +00:00",
      "deletedAt": null
    }
  ]
}
```

Tabla 1. Línea base de vulnerabilidades (entregable Fase 1).

#	Vulnerabilidad	OWASP Top 10	Endpoint
1	Inyección SQL en login	A03 Injection	POST /rest/user/login
2	XSS reflejado en búsqueda	A03 Injection	GET /rest/products/search?q=
3	IDOR en cesta/pedidos	A01 Broken Access Control	/rest/basket/:id
4	Exposición de hash de password	A07 / Sensitive Data	GET /rest/user/whoami, /api/Users
5	Path traversal / LFI	A01 / A03	GET ...q=../../etc/passwd

Fase 2 — Capa base WAF/WAAP (reglas CRS, PL1)

Bloqueo exitoso: payload SQLi clásico q=' OR '1'='1 responde 403 Forbidden. El log del proxy (docker compose logs waap-proxy) registra regla CRS 942100 "SQL Injection Attack Detected via libinjection" y 949110 "Inbound Anomaly Score Exceeded (5)". En esta imagen el log de auditoría se emite a docker logs (stderr), no a /var/log/modsec_audit.log.

Figura 4. Bloqueo CRS: SQLi clásico → 403 Forbidden (nginx).



Evasión exitosa (base comparativa para Fase 3): payload fragmentado en dos parámetros q=' & x=OR 1=1 responde 200 OK con

{"status":"success","data":[]}

 — el motor PL1 evalúa cada parámetro por separado y no detecta la inyección. Motivación directa para el módulo IA/ML y RASP.

Figura 5. Evasión: payload fragmentado → 200 OK (no detectado por CRS PL1).

Pretty-print ☐

```
{"status":"success","data":[]}
```

Tabla 2. Matriz Fase 2 (entregable: 1 bloqueo + 1 evasión).

Payload / técnica	Reglas CRS PL1	Evidencia
SQLi clásico ' OR '1'='1	BLOQUEADO (403)	Fig. 4 + reglas 942100/949110
SQLi URL-encoded	BLOQUEADO (403)	docker logs waap-proxy
SQLi comentarios /**/	BLOQUEADO (403)	docker logs waap-proxy
XSS <script>alert(1)</script>	BLOQUEADO (403)	reglas 941100/941110/941160
Path traversal ../../etc/passwd	BLOQUEADO (403)	reglas 930100/930110/930120
SQLi fragmentado q=' & x=OR 1=1	EVADE (200)	Fig. 5

Anexo — Salidas de terminal

```
===== docker compose ps =====
NAME                IMAGE                                COMMAND
SERVICE            CREATED              STATUS              PORTS
taller-waap-juice-shop-1  bkimminich/juice-shop:latest  "/nodejs/bin/node /jâ€¦"
juice-shop            32 minutes ago      Up 32 minutes       0.0.0.0:3000->3000/tcp,
```

```
[::]:3000->3000/tcp
taller-waap-waap-proxy-1  owasp/modsecurity-crs:nginx  "/docker-entrypoint.â€¦"  waap-
proxy  32 minutes ago  Up 32 minutes (healthy)  0.0.0.0:8080->8080/tcp, [::]:8080-
>8080/tcp
```

```
===== curl cÃ³digos =====
HOME via WAAP 8080: 200
search normal: 200
sqli clasico: 403
evasion fragmentada: 200
```

```
===== WAAP env =====
PARANOIA=1
BACKEND=http://juice-shop:3000
MODSEC_DISABLE_BACKEND_COMPRESSION=off
BLOCKING_PARANOIA=1
```

```
===== regla CRS (bloqueo) =====
34 "ruleId":"949110"
34 "message":"Inbound Anomaly Score Exceeded (Total Score: 5)"
33 "message":"Request content type is not allowed by policy"
1 "ruleId":"942100"
1 "message":"SQL Injection Attack Detected via libinjection"
```

Fase 3 — Módulo de detección basado en IA/ML

Tráfico normal simulado con waap/ml/gen_traffic.py (120 peticiones a ritmo humano, ~15 min, vía WAAP): búsquedas, home, producto, login, challenges. Sin ningún 403 del CRS (solo 401/404 benignos), por lo que el set es comportamiento normal limpio. Extracción literal del PDF (6 features) y entrenamiento literal (IsolationForest n_estimators=200, contamination=0.02, StandardScaler) → waap/ml/model.pkl (120 muestras; el .pkl está en .gitignore y se regenera con train_model.py).

Evidencia de entrenamiento:

Modelo entrenado y guardado en waap/ml/model.pkl (120 muestras)

Matriz de 10 peticiones (5 normales / 5 ataque, incluida la evasión fragmentada de Fase 2):

etiqueta	score	<-0.05	<0.0	esperado
busqueda comun	0.0287	normal	normal	normal
login valido	-0.1024	ANOMALO	ANOMALO	normal
home	-0.1112	ANOMALO	ANOMALO	normal
producto	-0.0141	normal	ANOMALO	normal
score-board	-0.0202	normal	ANOMALO	normal
SQLi clasico	-0.0253	normal	ANOMALO	ataque
SQLi encoded (Fase 2)	-0.0845	ANOMALO	ANOMALO	ataque
SQLi comentarios	-0.0428	normal	ANOMALO	ataque
evasion fragmentada (Fase 2)	-0.0095	normal	ANOMALO	ataque
XSS reflejado	-0.0759	ANOMALO	ANOMALO	ataque

--- Rafaga bot: 20 consultas rapidas (req_per_minute alto) ---

```
req_per_minute= 5: score=+0.1571 anomal(<-0.05)=False
req_per_minute= 30: score=+0.1690 anomal(<-0.05)=False
req_per_minute= 120: score=+0.1690 anomal(<-0.05)=False
```

Comparativa de umbrales (detección vs falsos positivos):

```
umbral -0.05: TP=2/5 FN=3/5 FP=2/5 TN=3/5 deteccion=40% FPR=40%
umbral +0.00: TP=5/5 FN=0/5 FP=4/5 TN=1/5 deteccion=100% FPR=80%
```

Discusión de falsos positivos (entregable PDF): con umbral -0.05 se detecta 2/5 ataques (SQLi encoded, XSS) con 2 FP (login válido, home); con umbral 0.0 se detecta 5/5 pero con 4 FP. La ráfaga bot apenas mueve el score (+0.16→+0.17): IsolationForest no ordena más allá del máximo de entrenamiento, así que req_per_minute por sí solo no cruza el umbral — limitación documentada y propuesta de mejora (Q7): regla híbrida tasa+score. La evasión fragmentada (q=' solo) es estadísticamente normal: ni CRS ni ML la ven; motivación de Fase 4.

Fase 4 — Integración RASP

Nota metodológica: el rasp_agent.py Flask del PDF no es integrable a Juice Shop (Node/Express). Se implementó app/rasp-demo/server.js en Express, puerto 1:1 del PDF (mismo SQLi_PATTERN, misma query final concatenada, mismo 403 + log). OpenRASP está inactivo desde 2022 y sin agente Node vigente, por lo que el middleware propio es la vía académica viable. app/rasp_agent.py queda como referencia literal del PDF.

Bloqueos (directo al demo, sin WAAP):

```
===== CASO 1: fragmentado (user="" pass="OR 1=1--") =====
RASP ON : 403
RASP OFF: {"status":"success","user":"*","note":"injection: full dump","rows":2}
===== CASO 2: UNION (familia Fase 2) =====
RASP ON : 403
RASP OFF: {"status":"success","user":"*","note":"injection: full dump","rows":2}
===== CASO 2b: ' OR '1'='1 string-form (hueco patron literal PDF) =====
RASP ON : {"status":"success","user":"*","note":"injection: full dump","rows":2}
RASP OFF: {"status":"success","user":"*","note":"injection: full dump","rows":2}
===== legitimo =====
RASP ON : 200
RASP OFF: 200
```

Logs del agente:

```
RASP: consulta SQL bloqueada en tiempo de ejecucion: SELECT * FROM users WHERE user=''
AND pass='OR 1=1--'
RASP: consulta SQL bloqueada en tiempo de ejecucion: SELECT * FROM users WHERE
user='admin' UNION SELECT * FROM users--' AND pass='x'
```

Lectura: el fragmentado (403 con RASP / dump total sin RASP) y el UNION (idem) confirman la tesis del PDF — el contexto de ejecución ve lo que el perímetro no. Hallazgo honesto: la forma string ' OR '1'='1 evade también el regex literal del PDF (solo cubre OR 1=1 numérico); fix propuesto: añadir OR '...'='...' al patrón.

Latencia con/sin RASP:

HTTP end-to-end (30 reqs alternadas con warmup, login legitimo):

RASP ON : n=30 media=2.39ms p50=1.41ms p95=8.14ms

RASP OFF: n=30 media=3.70ms p50=1.54ms p95=8.50ms

-> La diferencia ON-vs-OFF es ruido entre procesos (el regex es sub-microsegundo frente a ~2ms de HTTP); ver microbenchmark.

Microbenchmark en proceso (200k iteraciones, mismo patron SQLI_PATTERN del PDF):

constructor solo : 0.0 micro-s/req

con guard RASP : 0.2 micro-s/req

overhead real del guard: +0.1 micro-s/req

Conclusion: overhead del RASP despreciable (~0.01% de la latencia HTTP). Coherente con el 1-4% reportado por OpenRASP en el peor caso (PDF p.6: limitacion real es el acoplamiento por servicio, no la latencia).

Matriz comparativa por capas (avance Fase 6)

Payload / técnica	Reglas CRS PL1	IA/ML (umbral 0.0)	RASP
SQLi clásico	BLOQUEA 403	Detecta	Bloquea 403
SQLi encoded	BLOQUEA 403	Detecta	Bloquea (UNION/--/OR 1=1)
SQLi fragmentado q=' & x=OR 1=1	EVADE 200	No detecta	Bloquea 403
XSS <script>	BLOQUEA 403	Detecta	N/A (demo SQL)
' OR '1'='1 string-form	BLOQUEA 403	Detecta (0.0)	EVADE (hueco patrón)
Ráfaga bot	N/A	No cruza umbral	N/A